Promo Finder: ranked promos from a microdrama screenplay

Kuku AI Generalist assignment · Problem 1 (Find the Promos) · prototype: Python repo with run steps in `README.md`

1. Problem and why I picked it

An editor today watches every episode and picks 20–60 s promo moments by hand, one show at a time, while new shows ship every week. A good promo opens on a hook in its first 3 seconds, ends on an unresolved cliffhanger, and makes sense to a stranger. I chose this problem because its output can be checked: every promo is an episode plus start and end timestamps that an editor can open and judge in seconds.

2. Assumptions (where the brief was silent)

- The input is the screenplay text only: no video or audio. Script timestamps are assumed close enough to the video for an editor to fine-tune.
- The tool **proposes** and an editor **decides**. It ranks a shortlist of 30 promos with reasons and flags, not a single "best" clip.
- A promo has 1–3 segments, total length 20–60 s. Stitched segments come from the same or adjacent episodes, so they stay understandable.
- Variety matters when posting: at most 2 promos per episode, 1 per scene and 5 per block of 5 episodes, so the list is not one story arc repeated.
- Free tiers only, and the model must be swappable without code changes, as the brief asks.

3. Solution and core workflow

One command (`make run SHOW_DIR=…`) runs five stages and writes `promos.json`, `promos.csv`, `ranking.md` and `quality_report.md`. A free rules-only mode (`make run-rules`) needs no API key. A local read-only viewer (`make ui`) lets an editor step through the ranking, copy a promo's timestamps, download the CSV and read the quality report.

1. **Parse:** each episode file becomes a timeline of timestamped beats (scene, dialogue, action). Malformed lines produce warnings, not crashes.
2. **Generate candidates (code):** every 20–60 s window that starts and ends on a beat boundary (never mid-line) is a candidate (39,354 for this pack). Rule signals score the hook (first 3 s only), the cliffhanger and intensity from a keyword lexicon (slaps, reveals, threats, accusations, ending cues, resolution phrases). The tool removes overlapping windows (non-maximum suppression, overlap > 0.5), drops "previously on" recaps and builds a few stitched hook-plus-cliffhanger pairs. The top 8 per episode plus 40 stitched candidates go to the LLM.
3. **Score (LLM, cascaded):** batches of 8 candidates are scored 0–10 on hook, cliffhanger, intensity and spoiler risk against fixed anchors. The model answers only with *beat IDs* from the clip, never free-text timestamps, so it cannot invent a cut. The top 60 then get a *blind* clarity check: the model sees only the clip, as a stranger would. It can also pick a stronger unresolved end point, which is re-validated by code.
4. **Rank (code + LLM):** rule and LLM scores are blended, hard gates are applied, and caps and de-duplication enforce variety. A listwise LLM rerank of the top 30 runs twice, the second time with the order reversed to cancel position bias.
5. **Evaluate (code):** the quality report covers validation counts, score distributions, LLM-vs-rule disagreement, a rules-only baseline, two ablations, failure modes with real examples, and tokens.

4. Architecture, tools, and what is AI vs code

Part	Done by	Implementation
Parsing, candidate search, rule signals, caps, validation of every cut	Code	Python 3.12, Pydantic schemas; YAML config and keyword lexicon (`config/`). A cut that breaks a rule is never delivered.
Hook, cliffhanger, intensity, spoiler, blind clarity, end choice, rerank	LLM	Versioned prompts in `prompts/`; JSON answers validated, then retried on failure
Provider layer	Code	Gemini, Groq, Anthropic, OpenAI, Ollama, mock or none, chosen only via `.env`. Includes a response cache (re-runs resume and are not re-billed), a rate limiter with per-minute and per-day quotas, and call and USD budgets.
Final choice of what to post	Editor	Viewer and CSV; each promo carries reasons, flags and a confidence value

The model used for the delivered run was Gemini `gemini-3.5-flash-lite` (free tier). Swapping in a stronger model is a two-line `.env` change (`LLM_PROVIDER`, `LLM_MODEL`).

5. Identification and ranking method

```
dim   = 0.7·LLM + 0.3·rule            (clarity: 0.85·LLM + 0.15·rule)
score = 0.3·hook + 0.3·cliffhanger + 0.2·clarity + 0.2·intensity − 0.05·spoiler_risk
final = 0.5·score + 0.5·listwise rerank score       (top 30 only; stitched promos discounted by coherence)
```

- **Hard gates:** a promo is never delivered if the LLM scores its hook or its delivered ending below 5/10. The gates removed 117 of 428 candidates after the first pass and 2 more after blind scoring.
- **Post-first order** is the final score. Each promo states its hook beat, its cliffhanger beat, why it works, and why to post it early (its strongest dimensions).
- **Confidence** is higher when the rules and the LLM agree. Flags such as *needs context*, *weak ending*, *dead air* or *high spoiler risk* tell the editor what to check.

6. How the output answers the brief

Brief asks for	Where it is
Promos across all episodes, with the episode and start/end of every segment, in order	`segments[]` in `promos.json`/`.csv`; 30 promos drawn from 27 of the 50 episodes
The hook, the cliffhanger, one line on why it works	`hook`, `cliffhanger` (the beat, timestamp and line), `why`
Rank them: which to post first and why	`rank`, `post_first_because`, `ranking.md`
Check quality: where does it pick badly?	`quality_report.md` §9 and section 8 below

7. How I know it is good: testing and validation

Full run on the 50-episode pack (Gemini, rules + LLM):

- 3,727 beats parsed → 39,354 windows → 428 candidates scored → 60 fully scored → 30 delivered.
- 0 of 30 failed final validation; lengths run from 20.3 to 57.9 s.
- End refinement moved the end of 23 of the 30 promos to a stronger unresolved cut.
- The top 10 come from 10 different episodes.
- Rules vs LLM: Spearman correlation 0.46. On cliffhangers they differ by 4+ points 35% of the time, which is why the LLM carries most of the weight.

Robustness checks:

- **Rules-only baseline:** only 4 of its top 10 overlap the LLM's top 10. 7 of the rules' top 10 just end at an episode's end, against 1 of 10 for the LLM run.
- **Ablations:** removing the writer's literal "cliffhanger" annotations keeps 7/10 of the top 10 (moderate dependence). Removing the episode-end prior keeps 5/10.
- **Smoke runs** (3 episodes, real API): 10–12 calls; 74–105 output tokens per candidate; 0 invalid JSON and 0 truncated replies after the fixes.
- **Engineering:** 238 automated tests (mock LLM, synthetic scripts), 96% line coverage, lint and type checks, plus a scan that fails if any 8-word script passage is in a committable file.

What I have **not** measured: agreement with a human editor. `golden.yaml` (label about 10 promos, then the report scores precision@10) is built but not yet labelled. The temperature-0 stability re-score exists but was not run on the full pack.

8. Where it picks badly, limitations and next steps

- **Needs backstory:** the tool's own flags mark 23 of 30 promos as needing context for a stranger. This is the biggest weakness.
- **Spoilers:** 4 of 30 reveal an outcome a promo should only tease. **Dead air:** 3 of 30 contain a long silent stretch.
- **Script-only:** slaps, stings and faces are inferred from text, and the LLM judge favours loud melodrama, so a quiet but effective hook scores low.
- **Single judge:** one model scores everything. The flags above are the model's own judgement, not human-verified. None of the 40 stitched candidates made the final 30, so multi-moment promos are untested in practice.
- **A reporting bug I found and fixed late:** the delivered run's report states two figures wrongly. It says 14 of 30 promos end at an episode's end; the correct count, after end refinement, is 1 of 30. Its "tokens per candidate" line divides a phase total by zero live calls. The code is fixed and tested. The raw outputs were left unedited.

Scale and cost. Candidate generation is pure code: about 13 s for 50 episodes on an 8 GB laptop with no LLM. By construction one show needs about 70 LLM calls for the main ranking (428 candidates in batches of 8, stitched in batches of 5, clarity in batches of 6, 2 rerank passes), at about 2,200 input tokens per scoring call. That is within the 200 requests/day limit configured for the free-tier key used here, and the cache means an interrupted run resumes without re-billing. At catalogue scale it runs per show as a batch job; paid-model cost is calls × token price, capped by per-run budgets. **Next:** label a golden set with an editor; feed a short show premise into the blind clarity check to cut false "needs context" flags; try a stronger judge model through the same `.env` switch; track which posted promos actually perform and fit the weights to that data.

9. AI tools used, and one place they got it wrong

Claude Code (Claude Sonnet 5.5, with Claude Opus 5.5 for the final audit) wrote most of the code under my design decisions and review. The real run used Gemini. Each mistake and its fix is logged in `AI_USAGE.md` (36 entries). **One place it got it wrong:** it matched the model's candidate IDs case-sensitively. Gemini answered `"C1"` where the code expected `c1`. The mock LLM always answered in lowercase, so all 169 tests passed, but the first real smoke run rejected 11 of 36 answers. The fix was case-insensitive matching plus a regression test. The second smoke run had 0 rejections.